

API Automation Testing Framework/Tool Evaluation

Goal: Build a lightweight API automation framework to test both legacy APIs and microservices.

Tools /Framework	Details	Recommendations
Karate	Karate is a Gherkin-based API testing tool. Pricing/Free: Pricing Pros: • Easy to start with little coding in JS or java (good for non-programmers) • Karate features native JSON support and one can write JSON expressions right within feature files. • JSON validation and multi-thread parallel execution • Logs and Reporting Cons: • Karate uses its own scripting language. • There is no IntelliSense support in IDE. • Finding errors in code is not easy. • TestNG support deprecated. • Custom support for our needs may be more difficult to implement • Has no Auth Schemes out of the box.	
Tosca	Tosca Test Automation is a Tricentis platform for cloud-based E-2-E testing Pricing/Free: Pricing Pros: • Already an existing relationship with Tricentis due to qTest and JIRA • No scripting required • Efficiently and automatically scale parallel tests • Model-based testing to simulate API services Cons: • Limiting Licensing	
ACCELQ	ACCELQ is a cloud-based continuous testing platform that seamlessly automates API and web testing Pricing/Free: Pricing Pros: • No scripting required • Behavior Driven Development • API and UI Functional Automation Testing • API Test Case Management, Test Planning, Execution and tracking governance • CI Driven regression suite executions • Includes back office testing Cons: • Free trial but not open-source	

Katalon	Katalon is a comprehensive API, Web, Desktop Testing and Mobile testing tool for beginners and experts. Pricing/Free: Free license with paid support services (not an open source) Pros: • It offers record and playback feature for non-programmers. • The in-built templates, frameworks, integration to Jira, Git, Jenkins etc. features help in speeding up the test case creation time. User only has to focus on automation of the test scripts. • Intuitive test reports, logging mechanism and screenshot on failure Cons: • It only supports Groovy. • Relatively small community support. • It has lot of performance issues (tool freezes, lagging, issues with text and object verification within iframes is problematic) • Custom support for our needs may be more difficult to implement	
ReadyAPI	ReadyAPI is the platform for the functional, security, and load testing of RESTful, SOAP, GraphQL, and other web services. Pricing/Free: Pricing Pros: • Scriptless • Data driven testing. • API Reporting dashboard • Security and Performance testing • Run Test suites in parallel. • Property Transfer Cons: • Only supports Groovy script. • Licensing cost • Limited options available to generate data using Data source.	
Current Legacy Standalone Automations	Current automations are legacy only and are built specifically for a single partner/project. Code may be duplicated between them. Pricing/Free: Free Pros: • Has already set up existing test cases which can be run • Already using a lot of same technology we may want to use (REST Assured, Gerkin, Maven) • Will be easier to build in custom support and features for our needs • Some of the general setup has already been completed • Built on Java Cons: • Not built as a framework • Will require restructuring and fixing in order to convert to a reusable framework • Work necessary will likely break some of the tests and may require their rework as well • Likely duplicate code between different standalone automations • No reporting built into automation	

Current Microservice Automation	Current automation is new microservices relying on being able to start up the service on the fly and then executing all tests. Pricing/Free: Free Pros: • Tests create their own data so tests are easily repeatable and less fragile • Has support from dev and architecture team if necessary • Built with CI/CD integration in mind • Has some level of generating reporting • Built on java Cons: • Setup is currently based on streams and may not work with legacy API • Current execution will launch the associated service (endpoints) with specific configuration to allow environment independent runs which may not be supported in legacy format • Due to the way tests are written, will require some understanding how to add additional tests and/or require dev help	
PRIME (ASTROAPI /UNITEAPI)	Current setup based on Divya/Jihoon's code pushed into PRIME core to allow testing of API endpoints Pricing/Free: Free Pros: • Built on PRIME so gives access to a lot of already existing features including database access, data passing, reporting, data generation, file reading and creation, loading test case information, data validation • Built on java • Data validation has been simplified once setup • Already has setup to handle calling endpoints and saving response (via REST Assured) Cons: • Adding new endpoints and tests is complicated • Not lightweight for single purpose of API testing • May be hard to build into CI/CD pipeline if necessary • Has seen limited use and may require additional work	

REST-Assured + Hybrid Framework	REST Assured is an open-source Java Library for testing RESTful web services. Pricing/Free: Free Pros: • Helpful for both small and large automation frameworks, easy CI /CD integration • Large community support • It has a powerful Java-based API that leverages Hamcrest as the matcher tool. • It also has a simple set of methods that allows method chaining and Pseudo-Gherkin Syntax (supports the syntax of BDD) • Using Hybrid framework gives the flexibility to implement different authorization types or encryption or handle complicated JSON validations. • Will be easier to build in custom support and features for our needs cons: • Not beginner-friendly • Will take longer to build out necessary framework Some potential disadvantages of using Cucumber at API level testing framework. 1. Complexity: Cucumber introduces an additional layer of complexity to the testing setup and requires additional effort and time to create feature files, step definitions, and managing the integration with testing frameworks 2. Performance Overhead: Cucumber is an extra layer of abstraction between tests and the testing framework. This can result in some performance overhead, as the feature files need to be parsed and executed, and step definitions need to be matched and executed. 3. Maintenance Overhead: As the number of feature files and step definitions grows, maintaining and refactoring them can become challenging. Changes in the underlying API or test requirements may require updates in multiple places, making maintenance more time-consuming.	• Ready-made tools have user-friendly interface and low-threshold coding requirements. However, the pitfall here is that they usually address only general API testing issues. • Building a customized API framework gives more freedom to adjust the framework according to the project needs and not limited to capabilities of a tool and its existing plugins. • It is easy to customize by adding any library suitable and build a reporting standard. • Will have more flexibility in managing the test data. • Team has Java expertise. • Will have more flexibility to implement the test scripts having APIs that requires intricate logic. • API testing requirements are relatively straightforward, using a simpler testing framework might be more appropriate instead of an additional layer of complexity due to using Cucumber framework.
---------------------------------	---	---